

Kapittel 15

Via objekter til NoSQL

Læringsmål

- Forstå begrensningene ved relasjonsdatabaser og motivasjonen for objektreasjonelle databaser og NoSQL-databaser.
- Kunne lage brukerdefinerte datatyper i den objektreasjonelle databasen PostgreSQL.
- Kjenne til lagringsprinsipper og bruksområder for NoSQL-databaser.
- Kunne gjøre databasedesign rettet mot dokumentdatabaser.

Kritikk av relasjonsdatabaser

- Relasjonsdatabaser egner seg mindre godt for å håndtere komplekse datastrukturer
 - Geografiske InformasjonsSystemer (GIS)
 - Tekniske tegninger
 - Multimedia
 - Semistrukturerte dokumenter
 - Sosiale medier
 - Stordata samlet inn på «tingenes internett»

Hva var poenget?

- Behandling av kartdata viser behovet for **sammensatte, brukerdefinerte** datatyper.
- I objektorienterte databasesystemer og programmeringsspråk kan man definere sine egne datatyper med **tilhørende operasjoner**.

Objektrelasjonelle databaser

- På 1990-tallet ble det utviklet **objektdatabaser** for å håndtere komplekse datastrukturer, f.eks. kartdata.
- Som svar på dette ble relasjonsdatabasene utvidet med objektorienterte mekanismer:
 - Mulighet for å definere egne datatyper med tilhørende metoder, som kan brukes i tabelldefinisjon og SQL.
 - «Collection»-klasser: Tabell, sekvens, multiset.
 - Subklasser (spesialisering)
- Slike hybridsystemer kalles for **objektrelasjonelle databaser** og fikk større kommersielt gjennomslag enn de rene objektdatabasene.

Dat typer i PostgreSQL

Støtte for SQL og standard datatyper. Dessuten:

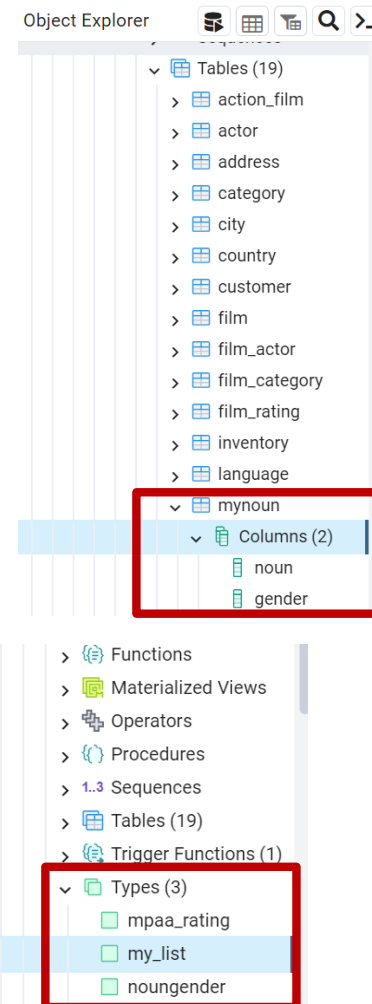
- Brukerdefinerte datatyper
- Domenetyper
- Array-typer
- Oppramstyper
- Intervalltyper
- Geometrityper
- XML- og JSON-datatyper

Brukerdefinerte datatyper

- Eksempler: kjør hver kommando separat (dvdrental db).

Query Query History

```
1  --Composite type
2  CREATE TYPE my_list AS (name text, description text);
3  CREATE FUNCTION show_films() RETURNS SETOF my_list AS $$ SELECT title, description FROM film $$ LANGUAGE SQL;
4  SELECT show_films();
5
6  --Enum type
7  CREATE TYPE nongender AS ENUM ('masculine', 'feminine');
8  CREATE TABLE mynoun (noun text, gender nongender);
9  INSERT INTO mynoun VALUES ('bok', 'feminine');
10 INSERT INTO mynoun VALUES ('stol', 'masculine');
11 ALTER TYPE nongender ADD VALUE 'neutral';
12 INSERT INTO mynoun VALUES ('hus', 'neutral');
```



Kilde 1: <https://www.postgresql.org/docs/current/sql-createtype.html>

Kilde 2: <https://www.postgresql.org/docs/current/sql-altertype.html>

Brukerdefinerte datatyper

Arv: vi har tabeller cities og capitals, hvor capitals er cities. Vi vil vise capitals på en implisit måte når vi viser alle cities.

```
CREATE TABLE cities (  
  name      text,  
  population real,  
  elevation int    -- (in ft)  
);
```

name	population	elevation
text	real	integer
Mariposa	1060	1953
Las Vegas	656274	2174
Madison	272903	845

```
CREATE TABLE capitals (  
  state      char(2) UNIQUE NOT NULL  
) INHERITS (cities);
```

name	population	elevation	state
text	real	integer	character
Madison	272903	845	WS

```
SELECT name, elevation  
FROM ONLY cities  
WHERE elevation > 500;
```

name	elevation
Las Vegas	2174
Mariposa	1953

(2 rows)

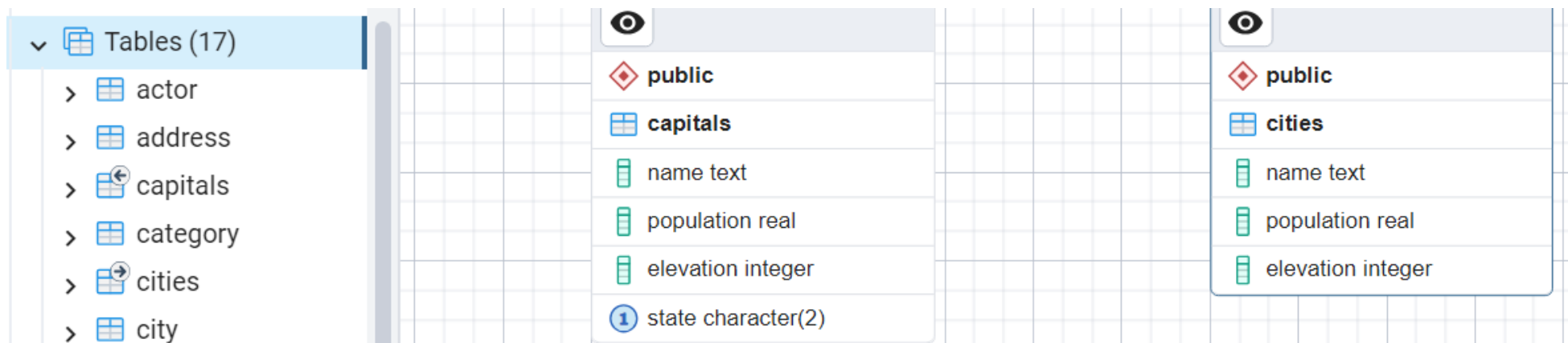
```
SELECT name, elevation  
FROM cities  
WHERE elevation > 500;
```

name	elevation
Las Vegas	2174
Mariposa	1953
Madison	845

(3 rows)

Brukerdefinerte datatyper

- Sånn ser arv i PostgreSQL:



Domenetyper

Tar utgangspunkt i en eksisterende datatype og begrenser lovlige verdier ved hjelp av en **CHECK**-regel:

```
CREATE DOMAIN AlderType AS SMALLINT  
CHECK (VALUE BETWEEN 0 AND 150);
```

AlderType kan nå brukes som datatype i **CREATE TABLE**, kanskje i flere ulike tabeller:

```
CREATE TABLE Student(  
... Alder AlderType, ...  
);
```

Arrays og oppramstyper

En **array** brukes for å ta vare på en liste med verdier av samme datatype, f.eks. en liste med telefonnumre.

Eksempel på kolonnedefinisjon: **Tlf TEXT[]**

For å få tak i første verdi: **Tlf[0]**

En **oppramstype** blir definert ved å liste opp alle verdiene:

```
CREATE TYPE StatusType AS  
    ENUM ('aktiv', 'permisjon', 'sluttet');
```

Id	Fornavn	Adresse	Tlf	Status
5	Ole	(Furulia, 28)	{'22334455','88776655'}	aktiv

NoSQL og Big Data

- **NoSQL** er en sekkebetegnelse på en rekke nyere alternativer til relasjonsdatabaser.
 - NoSQL = **Not Only SQL**
 - Komplekse datastrukturer
 - Semistrukturerte data
 - Distribuerte løsninger
 - Løse krav til transaksjonshåndtering
- **Big Data**: datasett som er så store og komplekse at de tøyser grensene for hva som er mulig å håndtere, både med hensyn til lagring, utveksling og behandling.
 - **Volume + Velocity + Variety + Veracity + Value**

Nøkkel/verdi-databaser (key/value)

- Data er organisert ved en samling av **nøkkel/verdi-par**, som f.eks. Fornavn='Ola'.
- Velegnet for applikasjoner som gjør mange, enkle oppslag basert på nøkler.

Nøkkel	Verdi
Elev:1:Fornavn	Ailin
Elev:1:Etternavn	Liane
Elev:1:Fødselsdato	09.10.2018
Elev:2:Fornavn	Gorm
Elev:2:Etternavn	Syrstad

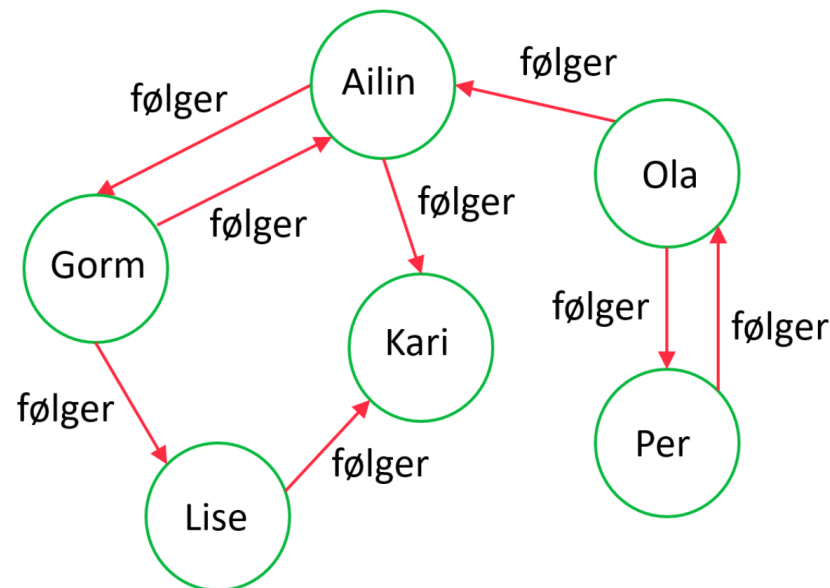
Dokumentdatabaser

- Data lagres som et antall **semistrukturerte dokumenter**, gjerne i form av **JSON** eller XML.
- Hvert dokument har en unik id (nøkkel).
- Velegnet som database bak webløsninger.
- Eksempler: MongoDB, DataBricks.

Id	Dokument
1	{ Fornavn: Ailin, Etternavn: Liane, Fødselsdato: 09.10.2018 }
2	{ Fornavn: Gorm, Etternavn: Syrstad }

Grafdatabaser

- En **graf** (et «nettverk») er bygd opp fra **noder** og **kanter**.
- Slike databaser egner seg for domener som er naturlig grafaktige, f.eks. **sosiale nettverk** – hvilke personer «følger» hvilke andre på Twitter.
- Eksempler: Neo4J, Microsoft Azure Cosmos DB.



Dokumenter

- Et **dokument** blir representert som et JSON-objekt og organiseres i **dokumentsamlinger**.
- En dokumentsamling kan sammenlignes med en tabell i en relasjonsdatabase.
- Et dokument består av et antall **felt**, og hvert felt er et **nøkkel/verdi-par**.

```
{  
  VNr: "10830",  
  Betegnelse: "Nisseskjegg",  
  Pris : 83.00,  
  Antall: 42  
}
```


Design av dokumentdatabaser

Eksempel: Varelageret i Hobbyhuset skal utvides med mulighet for å gi hver vare et antall beskrivende «kategori-tagger», og kundene skal kunne skrive kommentarer til en vare.

Mulig tabellstruktur i en relasjonsdatabase:

- Vare(VNr, Betegnelse, Pris, Antall, Hylle, KatNr★)
- Tagg(TaggNavn)
- TaggForVare(VNr★, TaggNavn★)
- Kommentar(Id, KNr★, VNr★, Melding, Dato)

Subobjekter / embedding

```
{
  "_id": "10830",
  "Betegnelse": "Nisseskjegg",
  "Pris": 83.00,
  "Antall": 42,
  "Hylle": "B12",
  "KatNr": 13,
  "Tagger": ["jul ", "klær"],
  "Kommentarer": [
    {
      "KNr": 5002,
      "Melding": "Meget bra skjegg, jula er i boks.",
      "Dato": "2024-11-29T00:00:00.000Z"
    },
    {
      "KNr": 5009,
      "Melding": "Solid nisseskjegg, men i lengste laget .",
      "Dato": "2024-12-18T00:00:00.000Z"
    }
  ]
}
```

- Både tagger og kommentarer er «innbakt» (**embedded**) i ordreobjektet.
- Taggene som en **array av tekststrenger**.
- Kommentarene som en **array av objekter**.

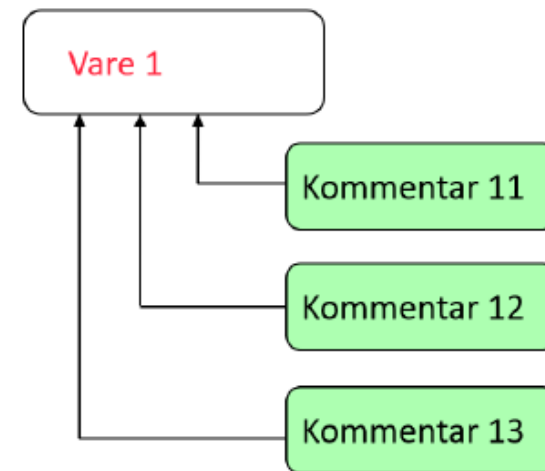
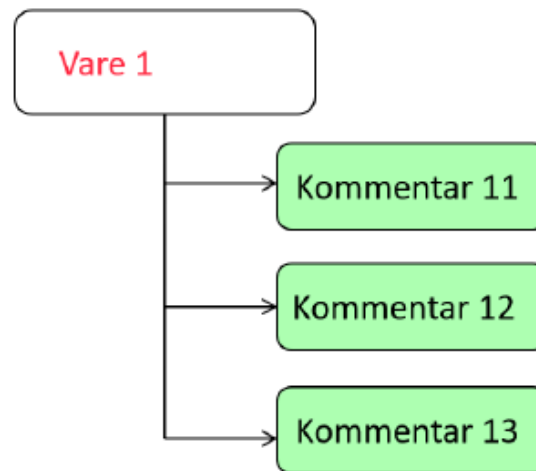
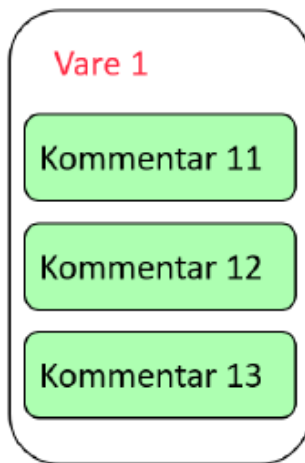
ER med sammensatte attributter

- Sammensatte attributter (objekter, arrays) kan presenteres på denne måten i et **ER-diagram** (utvidet med komplekse datatyper).

Vare
<i>_id</i> : ObjectId (PK)
Betegnelse : String
Pris : Number
Antall : Number
Hylle : String
KatNr : Number
Tagger: Array[String]
Kommentarer : Array[Object]
KNr : Integer
Melding : String
Dato : Date

Underobjekter eller referanser?

- Se på en-til-mange forholdet mellom Vare og Kommentar.
 - Hver vare utvides med en array av kommentarer (embedding).
 - Hver vare utvides med en array av referanser til kommentarer.
 - Hver kommentar utvides med en referanse til tilhørende vare.



JSON Dokumenter

JavaScript Object Notation (JSON): et åpent standard filformat og datautvekslingsformat som bruker menneskelig lesbar tekst til å lagre og overføre dataobjekter som består av attributt-verdi-par.

- Det er raskt og enkelt å analysere.
- Det er språkuavhengig med parsere for mange språk.
- Det brukes til å overføre data mellom en server og et webprogram.
- De grunnleggende datatypene er tall, string, boolsk, array, objekt osv.

[illegible]

Spørringer om JSON Data

- For å kjøre spørringer om JSON data så er det symboler som brukes.

QueryQuery History

1

--Querying a single value

2

SELECT data->>'price' AS "iPhone_price"

3

FROM json_data

4

WHERE data->>'name' = 'Apple Phone';

Data OutputMessagesNotifications

≡+📄▼📋▼🗑️🗄️⬇️📈

iPhone_price
text🔒

1200

QueryQuery History

1

--Querying an entire record

2

SELECT data FROM json_data;

Data OutputMessagesNotifications

≡+📄▼📋▼🗑️🗄️⬇️📈

data
jsonb🔒

1{"name": "Apple Phone", "type": "phone", "brand": "ACME", "price": 200, "available": true, "warranty_years": 1}

Spørringer om JSON Data

```
CREATE TABLE json_employee (  
  employee_id SERIAL NOT NULL PRIMARY KEY,  
  department VARCHAR(50),  
  extra_info JSON  
);
```

```
INSERT INTO json_employee (department,extra_info) VALUES (  
'accounting',  
,  
{  
  "ID": 12987654,  
  "fname": "John",  
  "lname": "Smith",  
  "Cities": ["Paris", "London", "Quito"]  
}  
);  
  
INSERT INTO json_employee (department,extra_info) VALUES (  
'management',  
,  
{  
  "ID": 12987655,  
  "fname": "Mary",  
  "lname": "Fox",  
  "email": ["mary@teamsql.io", "mary@fox.com"]  
}  
);
```

Spørringer om JSON Data

- >> Extracts and returns the element content as simple text
- > Extracts and returns the original JSON type (which might be an object)
- #>> Extracts and returns an array element as a simple text
- #> Extracts and returns an array element as a JSON object

```
SELECT employee_id,  
       extra_info->>'lname' AS Last_Name,  
       extra_info->>'fname' AS First_Name  
FROM json_employee;
```

	employee_id	last_name	first_name
	integer	text	text
1	1	Smith	John
2	2	Fox	Mary

```
SELECT employee_id,  
       extra_info->>'lname' AS Last_Name,  
       extra_info->>'fname' AS First_Name,  
       extra_info->'email' AS emails  
FROM json_employee;
```

	employee_id	last_name	first_name	emails
	integer	text	text	json
1	1	Smith	John	
2	2	Fox	Mary	["mary@teamsql.io", "mary@fox.com"]

```
SELECT employee_id,  
       extra_info#>'{email,1}' AS second_emails  
FROM json_employee;
```

	employee_id	second_emails
	integer	json
1	1	
2	2	"mary@fox.com"

Spørringer om JSON Data

```
ALTER TABLE json_employee ADD COLUMN full_name TEXT;
```

```
UPDATE json_employee  
SET full_name = (extra_info ->'lname')::text || ',' ||  
                (extra_info ->'fname')::text;
```

```
--View data
```

```
SELECT * FROM json_employee;
```

Output pane				
Data Output Explain Messages History				
	employee_id integer	department character varying(50)	extra_info json	full_name text
1	1	accounting	{ "ID": 12987654, "fname": "John", "lname": "Smith", "Cities": ["Paris", "London", "Quito"] }	"Smith", "John"
2	2	management	{ "ID": 12987655, "fname": "Mary", "lname": "Fox", "email": ["mary@teamsql.io", "mary@fox.com"] }	"Fox", "Mary"

Tilbakeblikk og trender

- **JSON Relational Duality:** Data lagres i normaliserte tabeller, men kan brukes/presenteres som JSON.
 - Et alternativ til ORM-løsninger
 - Nok et forsøk på å bygge «bro» mellom relasjonsdatabaser og objektorienterte programmeringsspråk
- Kanskje vi kan oppsummere med at **relasjonsdatabaser** fortsatt egner seg godt for **strukturerte data**, at ulike **NoSQL-databaser** egner seg godt for anvendelser som må håndtere **komplekse** og/eller **semistrukturerte data** – og at hierarkiske data i form av **JSON er på vei inn i objektreasjonelle databaser?**